

BMET5933 Assignment 2 — Demo Summary

Deep Learning Technical Contribution · James Wu

This is the slim demo notebook for the kidney CT classification project. It mirrors my PowerPoint deck section-by-section so I can fall back to it if I am not allowed to use slides during the demo. I keep only the meaningful code snippets — full implementations and exhaustive analyses live in

`James_Wu_Deep_Learning_Technical_Contribution.ipynb` and in the `Planning/` vault.

Project: 4-class kidney CT classification (Cyst / Normal / Stone / Tumor) on the Islam et al. 2022 benchmark. **Team:** Anthony Lee (classical handcrafted features + SVM) and James Wu (me, transfer-learned CNNs). **My contribution:** the entire deep-learning pipeline plus the shared preprocessing and the leakage diagnostic.

1. Shared infrastructure (what both pipelines use)

Both Anthony's classical SVM and my deep-learning models share the same data loader and the same evaluation harness. I designed these jointly with Anthony in Phase 0 so the classical-vs-DL comparison is apples-to-apples.

The key shared building block is `load_image()` — every CT slice enters the system through this function before any pipeline-specific work happens.

```
In [ ]: # Shared image loader – used by both classical and deep-learning pipelines
# I want to show this because every figure in my notebook ultimately depends
# on this one function producing identical inputs for both paradigms.

from PIL import Image
import numpy as np
from shared.config import IMAGE_SIZE    # (256, 256)

def load_image(path):
    # Step 1: open and convert to single-channel grayscale.
    # CT is intrinsically 1-channel intensity. The 3-channel replication
    # for ImageNet-pretrained CNNs happens later in my dataset code.
    img = Image.open(path).convert("L")

    # Step 2: square center crop to min(width, height).
    # The kidney is centered in every Islam-dataset slice, so cropping
    # to the central square preserves anatomy without introducing
    # the black bars that letterboxing would add.
```

```

w, h = img.size
m = min(w, h)
left = (w - m) // 2
top = (h - m) // 2
img = img.crop((left, top, left + m, top + m))

# Step 3: resize to the canonical 256x256 working resolution
# using bilinear interpolation (smooth, edge-preserving).
img = img.resize(IMAGE_SIZE, Image.BILINEAR)

# Return uint8 numpy. Downstream code does its own float casting.
return np.asarray(img, dtype=np.uint8)

```

Why this matters for my demo: the deep-learning input pipeline and the classical feature-extraction pipeline both start from the same 256×256 grayscale uint8 array. Differences from here on are intentional, not accidents of preprocessing.

2. Dataset and split

After deduplication (more on this in Section 8), the benchmark contains 11,929 unique CT slices across four classes. I work from the same `split_full.csv` that Anthony's pipeline uses — train/val/test = 8,146 / 1,895 / 1,888, stratified by class with patient-group buckets so adjacent slices from the same patient stay together.

```

In [ ]: # Just confirming the split I report against in this notebook.
import pandas as pd
from pathlib import Path

ROOT = Path.cwd()
if not (ROOT / "split_full.csv").exists() and (ROOT.parent / "split_full.csv").exists():
    ROOT = ROOT.parent

split = pd.read_csv(ROOT / "split_full.csv")
summary = (
    split.groupby(["split", "class"]).size()
    .unstack(fill_value=0)
    .reindex(columns=["Cyst", "Normal", "Stone", "Tumor"])
)
summary["Total"] = summary.sum(axis=1)
print(summary)

```

I deliberately do not touch the held-out test set during training or checkpoint selection — every test number I report is from a single inference pass on the final model.

3. Architecture choice — two CNN backbones

I picked two backbones for the deep-learning side: **EfficientNet-B0** (5M parameters, my baseline) and **ConvNeXt V2 Base** (89M parameters, my primary model). They share

the same data, same protocol, same evaluation harness, so the comparison isolates the architecture-and-pretraining effect.

I chose these specifically because:

- EfficientNet-B0 is small enough to fit comfortably in memory and trains fast, giving me a solid lightweight baseline.
- ConvNeXt V2 Base has ImageNet-22k → 1k pretraining and is a modern CNN that competes with vision transformers — I wanted to test whether more capacity and stronger pretraining help on this benchmark.

```
In [ ]: # My model builders. I keep a single dispatcher so I can swap backbones
# with one CLI flag (--model efficientnet_b0 or --model convnextv2_base).

import torch.nn as nn
from torchvision.models import EfficientNet_B0_Weights, efficientnet_b0
import timm

def build_efficientnet_b0(num_classes=4):
    # Load ImageNet-1k pretrained weights, then swap the 1000-class head
    # for a 4-class head with dropout 0.3 for mild regularisation.
    weights = EfficientNet_B0_Weights.IMAGENET1K_V1
    model = efficientnet_b0(weights=weights)
    in_features = model.classifier[1].in_features
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.3),
        nn.Linear(in_features, num_classes),
    )
    return model

def build_convnextv2_base(num_classes=4, image_size=384):
    # Pick the matching pretrained checkpoint for the requested input size.
    # I use the 384 variant because larger inputs give the model more spatial
    # detail (important for the small Stone calcifications).
    name = (
        "convnextv2_base.fcmae_ft_in22k_in1k_384"
        if image_size >= 384
        else "convnextv2_base.fcmae_ft_in22k_in1k"
    )
    return timm.create_model(
        name,
        pretrained=True,
        num_classes=num_classes,
        drop_path_rate=0.3,    # stochastic depth – key regularisation for 89
        drop_rate=0.3,        # head dropout
    )
```

4. Two-stage transfer learning

I train each backbone in two stages:

1. **Stage 1 (5 epochs)** — freeze the backbone, train only the new classifier head with Adam at LR $1e-3$. This lets the head reach a reasonable starting point without disturbing the ImageNet features.
2. **Stage 2 (60 epochs)** — unfreeze the last stage of the backbone, switch to AdamW with cosine learning-rate schedule (5-epoch warmup, peak $1e-5$, anneal to $1e-7$). Class-weighted cross-entropy compensates for the imbalance (Stone is roughly 3.7x rarer than Normal).

I save the checkpoint with the best validation macro-F1, not the final epoch.

```
In [ ]: # Skeleton of my two-stage training loop. Full implementation in
# deep_learning/train.py - I'm showing only the parts that matter for the de

from torch.optim import Adam, AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR, LinearLR, SequentialLR

# Stage 1: freeze everything, train only the head
freeze_backbone(model)
optimizer = Adam(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-3,
)
for epoch in range(5):
    train_one_epoch(model, train_loader, optimizer, loss_fn)
    val_f1 = evaluate_macro_f1(model, val_loader)
    if val_f1 > best_val_f1:
        torch.save(model.state_dict(), ckpt_path)

# Stage 2: unfreeze the last backbone block, fine-tune with cosine LR
unfreeze_last_blocks(model, n_blocks=1)
optimizer = AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-5,
    weight_decay=5e-2, # heavy WD for ConvNeXt (89M params); 1e-4 for EffNet
)
# Linear warmup for 5 epochs, then cosine anneal for the remaining 55
warmup = LinearLR(optimizer, start_factor=0.01, end_factor=1.0, total_iters=5)
cosine = CosineAnnealingLR(optimizer, T_max=55, eta_min=1e-7)
scheduler = SequentialLR(optimizer, [warmup, cosine], milestones=[5])

for epoch in range(60):
    train_one_epoch(model, train_loader, optimizer, loss_fn)
    val_f1 = evaluate_macro_f1(model, val_loader)
    if val_f1 > best_val_f1:
        torch.save(model.state_dict(), ckpt_path)
    scheduler.step()
```

5. Inference and test-time augmentation

At inference I do two things beyond a plain forward pass:

1. **Save full softmax probabilities** (not just argmax predictions), because keeping probabilities lets me do paired McNemar's tests, soft-vote ensembles, and ROC-AUC without retraining.
2. **Horizontal-flip TTA**: predict on the original image, predict on the flipped image, average the two softmax vectors before argmax. Free accuracy boost at the cost of 2x inference time.

I tested rotation TTA too — it actually hurt accuracy, so I stuck with hflip only.

```
In [ ]: # Inference + TTA. The key idea is that I average softmax distributions,
# not argmaxes. Averaging argmaxes throws away the model's confidence.

import torch
import numpy as np
from PIL import Image

@torch.no_grad()
def predict_with_tta(model, image_uint8):
    model.eval()

    # Build two views: original and horizontally flipped
    pil = Image.fromarray(image_uint8, mode="L")
    flip = pil.transpose(Image.FLIP_LEFT_RIGHT)

    # Pass each through the standard transform + model
    x_orig = transform(np.asarray(pil)).unsqueeze(0).to(device)
    x_flip = transform(np.asarray(flip)).unsqueeze(0).to(device)

    # Average softmax probabilities, then argmax
    p_orig = torch.softmax(model(x_orig), dim=1)
    p_flip = torch.softmax(model(x_flip), dim=1)
    p_avg = (p_orig + p_flip) / 2

    return p_avg.argmax(dim=1).item(), p_avg.cpu().numpy()
```

6. Headline numbers (clean held-out test set, n = 1,888)

I report macro-F1 because the class distribution is imbalanced (Stone ~13 %, Normal ~41 %) — accuracy would over-weight the easy majority class.

```
In [ ]: # Load the four pre-computed result JSONs and display a compact comparison.
import json
from pathlib import Path
import pandas as pd

RESULTS = ROOT / "Results"

models = {
    "EfficientNet-B0": "dl_run_full/dl_results.json",
    "EfficientNet-B0 + hflip TTA": "dl_run_full_tta_hflip/dl_results.json",
```

```

    "ConvNeXt V2 Base": "convnextv2_full_run/dl_results.json",
    "ConvNeXt V2 Base + hflip TTA": "convnextv2_full_run_tta_hflip/dl_results.json"
}

rows = []
for name, rel in models.items():
    r = json.loads((RESULTS / rel).read_text())
    rows.append({
        "Model": name,
        "Accuracy": round(r["accuracy"], 4),
        "Macro-F1": round(r["macro_f1"], 4),
        "Stone F1": round(r["per_class"]["Stone"]["f1"], 4),
    })

pd.DataFrame(rows)

```

ConvNeXt V2 + TTA is my best single deep-learning model at macro-F1 0.837. EffNet-B0 + TTA is 0.795. For comparison, Anthony's classical SVM reaches macro-F1 0.909 on the same test set — classical wins on this clean benchmark.

7. Per-class results — Stone is the universal weak class

Stone is the lowest-F1 class for every model I trained. The dominant error type is **Stone** → **Normal**, because small focal calcifications get diluted when a whole-slice CNN classifier pools over the entire image.

```

In [ ]: # Show the confusion matrix for my best DL model so the marker can see
        # where it fails.
import numpy as np
import matplotlib.pyplot as plt

r = json.loads((RESULTS / "convnextv2_full_run_tta_hflip" / "dl_results.json").read_text())
cm = np.array(r["confusion_matrix"])
classes = r["classes"]

fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(cm, cmap="Blues")
ax.set_xticks(range(4), classes, rotation=35, ha="right")
ax.set_yticks(range(4), classes)
ax.set_xlabel("Predicted")
ax.set_ylabel("True")
ax.set_title("ConvNeXt V2 + TTA - clean test")
for i in range(4):
    for j in range(4):
        color = "white" if cm[i, j] > cm.max() * 0.55 else "black"
        ax.text(j, i, str(cm[i, j]), ha="center", va="center", color=color)
plt.tight_layout()
plt.show()

```

8. The leakage diagnostic — my headline finding

While exploring the dataset I noticed our DL models were hitting accuracies that looked too good (EfficientNet-B0 at 0.94, ConvNeXt V2 at 1.000). I wrote a small script that MD5-hashes every standardised pixel array and compares the train, val, and test sets for collisions.

I found **5.1 % of test images were bit-for-bit identical to training images** — 96 out of 1,867 hash collisions. The dataset author independently confirmed the duplicates two days later and released a deduplicated version, which is the dataset I use everywhere else in this notebook.

The leakage gap is the most important number in the project:

- **EfficientNet-B0**: dropped 17.3 percentage points (0.940 → 0.768)
- **ConvNeXt V2 Base**: dropped 21.4 percentage points (1.000 → 0.822)
- **Classical SVM**: dropped only 8.4 percentage points (0.993 → 0.909)

Deep networks were memorising the duplicated images; classical handcrafted features were not. That's the strongest evidence I have that prior reported 99 %+ accuracies on this benchmark are inflated.

```
In [ ]: # The actual leakage detection – the heart of my contribution.
import hashlib
from collections import Counter

def md5_image(path):
    # Hash the *standardised* 256x256 pixel buffer, not the raw file bytes,
    # so JPEG re-encoding and metadata edits don't hide identical content.
    img = load_image(path)          # the shared loader from Section 1
    return hashlib.md5(img.tobytes()).hexdigest()

# In the actual diagnostic I ran this in parallel over ~12k images, then
# intersected the per-split hash sets to count collisions:
#
#     train_hashes = {md5_image(p) for p in train_paths}
#     test_hashes  = {md5_image(p) for p in test_paths}
#     duplicates   = train_hashes & test_hashes
#
# Result on the original Islam et al. 2022 release:
# 96 / 1,867 test images had a bit-identical training-set sibling (5.1 %)
```

9. Summary of what I built

In bullet form (this is roughly the closing slide of my demo):

- **Shared layer** I co-designed with Anthony: `load_image()`, `load_split()`, `bootstrap_ci()`, paired McNemar's test.
- **My deep-learning code**: EfficientNet-B0 and ConvNeXt V2 builders, two-stage transfer learning with cosine LR + warmup, class-weighted CE, inference with hflip

TTA, multi-seed ensembling, Grad-CAM interpretation.

- **My most novel finding:** the MD5 leakage diagnostic — discovered 5.1 % bit-identical train↔test duplication that previously-published 99 %+ accuracies were silently relying on.
- **Honest conclusion:** on the cleaned benchmark, Anthony's classical SVM beats both of my deep-learning models by 7 to 14 percentage points. Deep networks aren't free-lunch superior here; they're more leakage-sensitive and have lower sample efficiency.

The full technical detail and every artefact lives in

`James_Wu_Deep_Learning_Technical_Contribution.ipynb` and the `Planning/` vault. This summary notebook is just the demo-friendly cut.